



Security Audit Report

Anoma Q2 2026 - Generic Call Resource & Forwarder

Authors:

Last Revised:
03.07.2026

Carlos Rodriguez,
Miroslav Stefanovic

Contents

Audit overview	1
The project	1
Scope	1
Approach	1
Audit plan	1
Conclusions	2
System overview	3
Audit dashboard	5
Target summary	5
Engagement summary	5
Severity summary	5
Threat model	6
Assumptions	6
Properties	6
Implementation checklist	7
Findings	11
Miscellaneous recommendations	12
Appendix: Vulnerability classification	16
Disclaimer	20

Audit overview

The project

In June 2026, HeliAx engaged Informal Systems to perform a security audit of `forwarder-bases`, `generic-call-forwarder` and `generic-call-resource`.

Scope

The audit focused on evaluating the correctness and security properties of the smart contracts and the generic resource.

Approach

At Informal Systems, our security audits are grounded in a holistic and systematic methodology that prioritizes understanding the system as a whole before inspecting implementation details. Rather than focusing solely on code-level issues, we begin by constructing a comprehensive threat model to identify the system's key security properties and invariants—conditions that must always hold true to maintain safety and correctness.

Our approach combines in-depth manual review with formal verification techniques, fuzzing, and testing tools to rigorously assess the system's design and implementation. This layered strategy enables us to uncover both architectural vulnerabilities and low-level defects.

The final deliverable includes:

- A documented threat model outlining key assumptions, attack surfaces, and critical invariants.
- A list of findings, each including a description of the issue, its impact, and actionable remediation recommendations.

This methodology ensures that our audits not only highlight specific bugs but also help teams understand and reinforce the security posture of their entire system.

For important legal disclaimers and limitations regarding the scope, use, and interpretation of this report, please refer to the **Disclaimer** section.

Audit plan

The audit was conducted between June 15th, 2026, and June 19th, 2026 by the following personnel:

- Carlos Rodriguez
- Miroslav Stefanovic

Conclusions

The Heliac team has delivered a well-structured and clearly written codebase with clean separation of responsibilities between the `forwarder-bases` contracts, the `generic-call-forwarder`, and the `generic-call-resource`. No security vulnerabilities were identified during the audit. The findings are limited to code improvement recommendations aimed at increasing readability and maintainability, and at tightening preconditions — both by catching invalid inputs earlier (at proving time rather than on-chain) and by restricting the invocation context of certain functions so they can only be called in the narrow circumstances for which they were designed, such as limiting signature validation and callback registration to within an active batch execution. As a general deployment recommendation, we advise the team to enforce multisig control for any account holding administrative privileges, including the emergency committee and the authority authorised to perform contract upgrades. Using a single EOA for these roles creates an undue dependency on a single private key for operations that are consequential and difficult to reverse.

System overview

The Anoma Resource Machine (ARM) is a stateless execution model run by every node that processes transactions. Users submit intents to the intent gossip network as unbalanced ARM transactions; solvers complete them into balanced transactions, which are then ordered and executed by the executor node to update global state. The atomic unit of state is a resource (immutable, created once and consumed once). Active system state is the set of resources that have been created but not yet nullified. Transactions group resources into actions sharing the same execution context, and each action is broken down into compliance units, each pairing one consumed and one created resource.

The **GenericCall** resource is an ephemeral resource kind designed to carry arbitrary EVM execution inside an ARM action. It commits to a target forwarder address and an exact array of EVM calls, and its ZK circuit proves those commitments are consistent. It has no authorization mechanism of its own (it can be implicitly authorized by placing it in the same action as another resource that requires the action tree root to be signed). On-chain, the Protocol Adapter processes the ARM transaction by verifying all ZK proofs and, for each resource whose logic proof passes, dispatching the resource's external call payload to the corresponding forwarder. The **forwarder-bases** contracts define the shared trust boundary for all forwarders: only the Protocol Adapter can call them. The **GenericCallForwarder** is the concrete forwarder for the generic call resource kind (a stateless public smart account that executes the committed call array against arbitrary EVM targets).

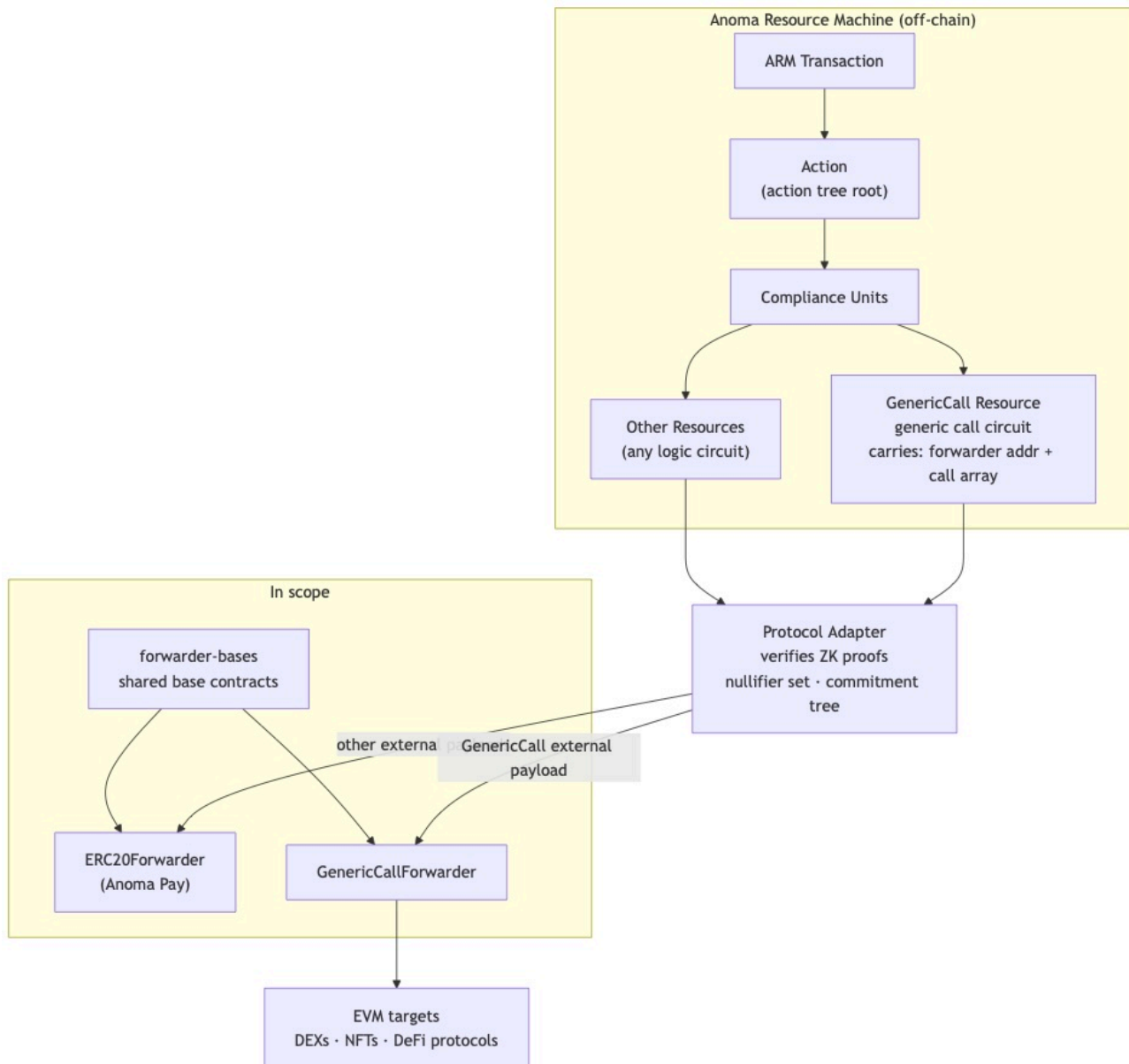


Figure 2: Components diagram

Audit dashboard

Target summary

- **Type:** Implementation
- **Platform:** Solidity, Rust
- **Artifacts:**
 - `forwarder-bases` at commit `7237c83`.
 - `generic-call-forwarder` at commit `64c3649`.
 - `generic-call-resource` at commit `4d561b1`.

Engagement summary

- **Dates:** June 15th, 2026 → June 19th, 2026
- **Method:** Threat modeling, manual code review

Severity summary

Finding Severity	Number
Critical	0
High	0
Medium	0
Low	0
Informational	1
Total	1

Table 1: Identified Security Findings

Threat model

Assumptions

1. The Protocol Adapter, the `arm-risc0` proof system, and the RISC0 verifier are correctly implemented and behave as specified: the PA only dispatches forwarder calls on valid proofs, and proofs are sound with respect to their circuit image IDs.

Properties

Forwarder

- Property FW-01: The forwarder returns the ERC-1271 magic value unconditionally for any caller, any message hash, and any signature.

This check holds. By design, having in mind that all transactions should be “atomic” and that `GenericCallForwarder` should not hold any funds after a transaction, this does not represent a threat.

- Property FW-02: The forwarder maintains a single shared asset pool; all ETH and tokens held by the forwarder are accessible to any authorized invocation regardless of which party deposited them.

This check holds. The intention of the design is not for `GenericCallForwarder` to hold tokens, but to forward them within a single transaction received from the Protocol Adapter. Having tokens retained on the smart contract after a transaction, together with unconditional return of the magic value, might cause an issue (see recommendation “Allow calls to `isValidSignature` only during a generic call batch”).

- Property FW-03: Persistent EVM side effects set within a batch (e.g. as ERC-20 approvals or allowances) survive beyond the transaction and remain in effect for any future interaction with the approved contract.

This check holds. Having `GenericCallForwarder` as an approved address might cause an issue with ERC-1271 implementation; this constraint should be enforced by the protocol adapter, as it is considered to be a source of truth. See also recommendation “Allow calls to `isValidSignature` only during a generic call batch” to limit the impact of validating signatures outside of a batch execution.

- Property FW-04: There is no function to withdraw assets from the forwarder except through an authorized call batch that explicitly transfers them.

This check holds. There is a recommendation to implement the possibility of a withdrawal function in case of collected dust (see recommendation “Add enforcement of withdrawal mechanism on inheriting contracts of `NativeTokenReceiver`”).

- Property FW-05: Callback selector mappings registered via `registerSelector` are stored in transient storage and do not persist beyond the current EVM transaction.

This check holds. `GenericCallForwarder` extends `TransientFallbackHandler` and does not store any `registerSelector` data beyond the current EVM transactions.

- Property FW-06: Registering a callback selector with a zero magic number is equivalent to unregistering it; any callback arriving with that selector will revert.

This check holds. Zero magic number represents an unregistered selector, if zero magic number is sent, the transaction will revert with `UnregisteredSelector` error ([ref](#)).

- Property FW-07: Emergency call forwarding is available only while the PA is paused; the emergency caller is designated at most once and cannot be changed after assignment.

This check holds. Calling `forwardEmergencyCall` will revert with `ProtocolAdapterNotStopped` error ([ref](#)) if the protocol adapter is not paused. Setting the emergency caller is only possible by the emergency committee and only once. There is a recommendation to consider implementing the possibility of updating emergency caller (see recommendation “Enable update of emergency caller”).

Circuit

- Property CR-01: A `GenericCall` resource is valid only if: (1) the ephemeral flag is true; (2) the label equals the hash of the target forwarder address; (3) the value equals the hash of the ABI-encoded call sequence; (4) a nullifier key is present when the resource is consumed.

This check holds. The ZK circuit constrains that the ephemeral flag is true ([ref](#)), that `label_ref` equals the hash of the forwarder address ([ref](#)), and that `value_ref` equals the hash of the ABI-encoded EVM calls to execute ([ref](#)), and that a nullifier key is present when the resource is consumed ([ref](#)).

- Property CR-02: The quantity field of a `GenericCall` resource is not constrained by the circuit.

This check holds.

Implementation checklist

Access control

- ☒ Constructor rejects a zero PA address

This check holds. All constructors have implemented checks for a zero address. There are two recommendations: expanding the error with a parameter name in case of a zero address (see recommendation “Add clarifying information for `ZeroNotAllowed` error”), and adding checks if the address is a smart contract for parameters that are expected to be smart contracts (see recommendation “Check if protocol adapter address is a smart contract”).

- ☒ `forwardCall` verifies both `msg.sender == _PROTOCOL_ADAPTER` and `logicRef == _LOGIC_REF` before any state change or external call

This check holds. Before changing any state, `forwardCall` function verifies that the protocol adapter is the function caller ([ref](#)) and that correct `logicRef` is sent ([ref](#)).

- ☒ `forwardEmergencyCall` is callable only by the designated emergency caller and only while the PA is paused

This check holds. Calling `forwardEmergencyCall` is only possible for a designated `emergencyCaller` (ref) that has been set by the emergency committee.

- ☒ `setEmergencyCaller` is callable only by the emergency committee, only once, and only while the PA is paused

This check holds. The implementation of `setEmergencyCaller` function only allows it to be called by the emergency committee (ref) that is set during contract deployment. Once the `emergencyCaller` is set, it can not be updated anymore, and this includes the emergency committee. There is a recommendation to consider the possibility of implementing a way to update the `emergencyCaller` (see recommendation "Enable update of emergency caller"). By design `emergencyCaller` can be set only when the protocol adapter has been paused.

Function visibility

- ☒ No internal function is unnecessarily exposed as public or external

This check holds.

Input validation

- ☒ An empty call array is rejected, or its behavior is explicitly defined

This check partly holds. In all but one case, it is handled correctly. In case of `_forwardCall` function in `GenericCallForwarder` an event will be triggered even if the array of `calls` is empty (see recommendation "Do not silently accept empty `calls` array").

- ☒ A call to a zero address is rejected, or its behavior is explicitly defined

This check partly holds. Sending tokens to a zero address is controlled by the protocol adapter that is considered a source of truth and is not checked in `GenericCallForwarder` by design, since the protocol adapter is the only address that can call `forwardCall` function (ref).

- ☒ Call with no data and no value produces a clear, documented error

This check holds. In `_forwardCall` function in `GenericCallForwarder`, in case of no data and no value, `NoOpNotAllowed` error is triggered.

- ☒ No unbounded batch size that could cause a call sequence to permanently exceed block gas limits

This property partly holds. The size of the batch is determined by the protocol adapter that is considered a source of truth and is not checked in `GenericCallForwarder` by design, since the protocol adapter is the only address that can call `forwardCall` function. However, the number of element in the `calls` array could be limited in the constrain logic of the generic call resource (see recommendation "Add size check of `calls` array").

Reentrancy

- ☒ `forwardCall` is protected by a reentrancy guard

This check holds. `forwardCall` function is decorated with modifier `nonReentrant` ([ref](#)).

- ☒ `registerSelector` and the ETH receive handlers are either protected or their unguarded accessibility is explicitly accepted

This check holds. All ETH or tokens received are being transferred or in case they are held on the `GenericCallForwarder`, available to the next caller by design.

UUPS upgradeability (`ForwarderBaseUpgradeable`)

- ☒ Implementation constructor calls `_disableInitializers()`

This check holds ([ref](#)).

- ☒ `_authorizeUpgrade` has a correct access control modifier

This check holds. We recommend to use multisig for an address that can initialize an upgrade.

- ☒ Upgrade target must implement the UUPS interface; upgrading to a non-UUPS implementation permanently removes upgrade capability

This check holds. `ForwarderBaseUpgradeable` uses OpenZeppelin v5 that checks for this condition and in case non-UUPS implementation is sent, it is reverted.

- ☒ All mutable states in `ForwarderBaseUpgradeable` is declared inside the ERC-7201 namespaced storage struct

This check holds. `ForwarderBaseUpgradeable` uses `_protocolAdapter` and `_logicRef`, and they are declared within a namespaced storage struct ([ref](#)).

- ☒ The ERC-7201 storage slot is correctly derived:
`keccak256(abi.encode(uint256(keccak256(namespace)) - 1)) & ~bytes32(uint256(0xff))`

This check holds.

Emergency recovery (`EmergencyMigratableForwarderBase`)

- ☒ The PA address implements `Pausable` interface as required by `setEmergencyCaller` and `forwardEmergencyCall`

This partly holds. Calling `setEmergencyCaller` checks if the protocol adapter has been paused; in case the provided protocol adapter does not support it, the call will fail. A smart contract does not check the interface of the protocol adapter when the address is set. There is a recommendation to validate that the protocol adapter address passed to the constructor is in fact an address of a smart contract (see recommendation "Check if protocol adapter address is a smart contract").

- ☒ `GenericCallForwarder` has no emergency recovery path

This check holds. There is the `EmergencyMigratableForwardedBase` smart contract that extends `ForwarderBase`, but `GenericCallForwarder` extends `ForwarderBase` directly.

Token and callback handling

- ☒ `registerSelector` correctly stores the magic number in transient storage, and the fallback correctly retrieves and returns it

This check holds ([ref](#)).

- ☒ The ETH `receive()` function accepts ETH unconditionally, and no logic inside it could consume more than the 2300 gas stipend when called from a `transfer` or `send`

This check holds ([ref](#)).

ERC-1271

- ☒ `isValidSignature` unconditionally returns the magic value

This check holds. `isValidSignature` function will always return magic number by design ([ref](#)).

Encoding boundary

- ☒ Call value encoding: 128-bit (Rust) to 256-bit (Solidity) zero-extension is semantically correct

This check holds. `U256::from(u128)` ([ref](#)) is a lossless widening of an unsigned integer.

Error handling

- ☒ Invalid forwarder address length prevents proof generation

This check holds. `ForwarderCallldata::from_bytes` ([ref](#)) enforces that the forwarder address is exactly 20 bytes. In `AnomaPay` the forwarder address is always sourced from a hardcoded configuration, so this path is unreachable in practice.

Findings

Name	Type	Severity	Status
Miscellaneous recommendations	Implementation	0 - Informational	Resolved

Table 2: Identified Security Findings

Miscellaneous recommendations

Severity **Informational**Exploitability **N/A**Status **Resolved**Type **Implementation**Impact **N/A**

forwarder-bases

- **Check if protocol adapter address is a smart contract:** The protocol adapter is always a smart contract, but the constructor of `ForwarderBase` only checks that `protocolAdapter` parameters is not the zero address. Consider checking as well that `protocolAdapter` is a smart contract.

Resolution: The development team decided against implementing this recommendation because the guarantees of this check are weak, since the smart contract might not be a protocol adapter.

- **Tighten access control for `registerSelector`:** Consider adding a `msg.sender == address(this)` guard to `registerSelector` (ref). Currently the function can be called by any address at any time (though in practice this poses no real risk since transient storage resets between transactions and the worst outcome is a batch revert). The intended usage is a self-call: a batch entry with `to = address(GenericCallForwarder)` registers a callback selector before the next call triggers it. Restricting to `msg.sender == address(this)` makes this intent explicit and prevents any other contract within the batch from interfering with callback registrations.

Resolution: The development team addressed this recommendation in commit `f8937ad`.

- **Enable update of emergency caller:** By design there is no revocation or update mechanism of the emergency caller: Once `_emergencyCaller` is set (ref), it cannot be changed or revoked. Consider allowing the emergency committee to be able to update the emergency caller.

Resolution: This is an intentional design choice.

- **Add clarifying information for `ZeroNotAllowed` error:** The `ZeroNotAllowed` error has not field that could help distinguish what `address` parameter causes the error in case there are multiple input addresses. Consider adding a field to the error to specify what parameter causes the error.

Resolution: The development team addressed this recommendation in commit `95513b7`.

- **Add events for emergency operation:** `setEmergencyCaller` (ref) permanently sets `_emergencyCaller` (the address authorized to invoke `forwardEmergencyCall`). The function assigns `_emergencyCaller = emergencyCaller` (ref) but emits no event. Consider emitting an event to produce an observable on-chain signal of the change.

Resolution: The development team addressed this recommendation in commit `4e00635`.

- **Unnecessary `calldata-to-memory` copy in `_handleFallback`:** The `fallback` function receives bytes `calldata` data and passes it to `_handleFallback(msg.sig, data)`, which accepts bytes `memory` data (ref). Data is then only passed to `FallbackHandled`, so in the signature of `_handleFallback data` parameters could also be `calldata` instead of `memory`.

Resolution: The development team addressed this recommendation in commit [f0b1871](#).

- **Add enforcement of withdrawal mechanism on inheriting contracts of `NativeTokenReceiver`:** The development team acknowledged the possibility that some dust could remain on the `GenericCallForwarder` contract. Over time this dust might increase in value and it might be beneficial to have a withdrawal method. Consider introducing an abstract withdrawal function to force implementors to acknowledge the requirement at compile time, or at minimum promote the comment to a NatSpec `@dev` warning on the contract.

Resolution: The development team added a `@dev` NatSpec command in commit [b86aa70](#) and added an `ISweepable` interface and `SweepableNativeTokenReceiver` base contract in commit [d6e16fb](#).

- **Reorder checks in `setEmergencyCaller`:** Consider reordering the precondition checks in `setEmergencyCaller` ([ref](#)) so the function caller should be checked first (emergency committee), then checking if the PA has been paused, and to be followed by checking if input is zero address, and finally check that emergency caller has not already been set.

Resolution: The development team addressed this recommendation in commit [a9b125f](#).

- **Remove double check in `forwardEmergencyCall`:** The function `forwardEmergencyCall` ([ref](#)) first checks if the `emergencyCaller` is set, and then checks if the message sender is `emergencyCaller`. If `emergencyCaller` is not set, the default value will be zero address, and no transaction could origin from zero address, therefore, in case the `emergencyCaller` is not set, the second condition will always fail, so the first check is not necessary logic wise. The only reason to keep it would be to have a more precise error. Consider removing the `emergencyCaller` set check ([ref](#)).

Resolution: The development team addressed this recommendation in commit [45b2f3c](#).

- **Emit events during initialization for `_protocolAdapter` and `_logicRef`:** Consider emitting events when `_protocolAdapter` and `_logicRef` are being set ([ref](#)).

Resolution: The development team decided implementing this recommendation because this event would only be emitted a single time and getters exist that can be called when the OpenZeppelin event `Initialized(uint64 version)` is emitted.

- **Use modifiers for access control checks:** When a function should be called by a specific address, consider replace the `require` statement with a modifier. This applies to the `_emergencyCaller` check in `forwardEmergencyCaller` ([ref](#)), the `_EMERGENCY_COMMITTEE` check in `setEmergencyCaller` ([ref](#)), and the `_PROTOCOL_ADAPTER` check in `forwardCall` ([ref](#)).

Resolution: The development team addressed this finding in commit [3658663](#).

- **Implement two-step ownership transfer:** `ForwarderBaseUpgradeable` uses `OwnableUpgradeable` ([ref](#)), which transfers ownership in a single step: calling `transferOwnership(newOwner)` takes effect immediately with no confirmation from the new owner. A mistaken or malicious call with a wrong address permanently transfers control with no recovery path. Replacing `OwnableUpgradeable` with `Ownable2StepUpgradeable` ([ref](#)) adds a two-step handoff: `transferOwnership` only nominates a pending owner, and ownership transfers only once the new owner calls `acceptOwnership`. The change requires updating the import, the inheritance list, and replacing `__Ownable_init_unchained`

with `__Ownable2Step_init` in `__ForwarderBaseUpgradeable_init`. The `onlyOwner` guard on `_authorizeUpgrade` is unchanged.

Resolution: The development team decided against implementing this recommendation to reduce the workload on the protocol governance that would need to accept the ownership. Instead, they will remove ownership from the deployer wallet immediately and will check the addresses carefully after each deployment.

- **Corrent event name in NatSpec:** The comment in `NativeTokenReceiver` (ref) states `NativeTokenDeposited` while the event actually emitted is `NativeTokenReceived`.

Resolution: The development team addressed this recommendation in commit `d4ab3e8`.

- **Fix NatSpec for UnregisteredSelector error magicNumber parameter:** The description of the `magicNumber` parameter (ref) implies that this field contains the desired magic number. In reality, when this error is thrown, `magicNumber` is always `bytes4(0)`.

Resolution: The development team addressed this recommendation in commit `bc641aa`.

- **Clarify NatSpec of _handleFallback to not imply an inheritance pattern the code does not permit:** The documentation (ref) suggests that subclasses of `TransientFallbackHandler` should override `fallback` and call `_handleFallback` from within it. However `fallback` is not marked virtual, so inheriting contracts cannot override it.

Resolution: The development team addressed this recommendation in commit `243d434`.

generic-call-forwarder

- **Do not silently accept empty calls array:** If `calls.length == 0` in `_forwardCall` (ref), the loop body never executes, yet the function emits `Executed([], [])` and returns `""` successfully. In the case where `data.length == 0 && value == 0` the function correctly reverts with `NoOpNotAllowed`, but if the array of `calls` is empty the event is still triggered. Consider emitting the event only for non empty `calls` and non `data.length == 0 && value == 0` use cases. Alternatively, consider disallowing in `constrain` function of `GenericCallWitness` empty `encoded_calls`.

Resolution: This is an intentional design choice.

- **Allow calls to isValidSignature only during a generic call batch:** `isValidSignature` (ref) returns the ERC-1271 magic value unconditionally for any `(hash, signature)` pair, ignoring both arguments. The function is `external pure` and callable by anyone at any time, including outside an active Anoma transaction. Consider implementing a transient execution flag: Set a flag at the start of `_forwardCall` and clear it at the end. Make `isValidSignature` a `view` function that reverts unless the flag is set. This ensures ERC-1271 calls can only succeed during an active Anoma batch execution, with no persistent on-chain state that an attacker might exploit after the batch completes.

Resolution: While the development team acknowledges this is a good suggestion, but it would prevent use cases such as wrapping tokens back into ERC20 resources using the ERC20 forwarder, which requires a signature request from the `Permit2` contract to be accepted on the generic call forwarder (ref) outside of a generic call.

- **Fix wrong contract title in constructor of `GenericCallForwarder`:** The comment ([ref](#)) reads “ERC-20 forwarder contract” but should read instead “generic call forwarder”.

Resolution: The development team addressed this recommendation in commit [a55d654](#).

generic-call-resource

- **Add zero address check in constrain logic:** `encode_generic_call_forwarder_input` ([ref](#)) converts the `to` field of each call from raw bytes to an `alloy_primitives::Address` using `try_into`, which only validates that the slice is exactly 20 bytes long. A zero address passes this check and is encoded into the external payload without error at proving time. On-chain, the outcome depends on the accompanying fields: if `data` is non-empty, `GenericCallForwarder` routes through `functionCall` ([ref](#)) or `functionCallWithValue` ([ref](#)) which revert because address zero has no code, wasting all gas consumed up to that point; if `data` is empty and `value` is non-zero, the call routes through `sendValue`, which succeeds (the ETH is sent to address zero and permanently burned).

A zero-address check could be added inside `encode_generic_call_forwarder_input` immediately after the `try_into` conversion, so the issue is caught at proving time with a descriptive error rather than discovered through an on-chain failure. Whether to apply the check unconditionally depends on whether intentional ETH burns to address zero are a supported use case. If they are, the restriction can be narrowed: reject zero address only when `data` is non-empty: since a call with `calldata` to address zero will always revert on-chain regardless and permit it when `data` is empty and `value` is non-zero, treating that combination as an explicit burn. If ETH burns are not a supported use case, a flat rejection of any zero `to` address is the simpler and safer choice.

Resolution: The development team decided against implementing this recommendation to avoid increasing proving times.

- **Add size check of `calls` array:** Currently `constrain` function places no upper bound on the number of entries in the `calls` array. Since the array is committed to via the ZK proof, only the user constructing the witness can submit an oversized batch (but there is nothing preventing them from doing so accidentally). The consequence is that the transaction passes proof verification and reaches `forwardCall` on-chain, only to run out of gas iterating over the calls and revert. Adding a sensible maximum calls count check at the start of `constrain()` would catch this at proving time with a descriptive `ProveFailed` error, failing fast before any on-chain resources are consumed.

Resolution: The development team accepts that generic calls can cause transactions to run out of gas, since it's not possible to define a meaningful upper bound for the calls array length.

Appendix: Vulnerability classification

For classifying vulnerabilities identified in the findings of this report, we employ the simplified version of [Common Vulnerability Scoring System \(CVSS\) v3.1](#), which is an industry standard vulnerability metric. For each identified vulnerability we assess the scores from the *Base Metric Group*, the [Impact score](#), and the [Exploitability score](#). The *Exploitability score* reflects the ease and technical means by which the vulnerability can be exploited. That is, it represents characteristics of the *thing that is vulnerable*, which we refer to formally as the *vulnerable component*. The *Impact score* reflects the direct consequence of a successful exploit, and represents the consequence to the *thing that suffers the impact*, which we refer to formally as the *impacted component*. In order to ease score understanding, we employ [CVSS Qualitative Severity Rating Scale](#), and abstract numerical scores into the textual representation; we construct the final *Severity score* based on the combination of the Impact and Exploitability sub-scores.

As blockchains are a fast evolving field, we evaluate the scores not only for the present state of the system, but also for the state that deems achievable within 1 year of projected system evolution. E.g., if at present the system interacts with 1-2 other blockchains, but plans to expand interaction to 10-20 within the next year, we evaluate the impact, exploitability, and severity scores wrt. the latter state, in order to give the system designers better understanding of the vulnerabilities that need to be addressed in the near future.

Impact Score

The Impact score captures the effects of a successfully exploited vulnerability on the component that suffers the worst outcome that is most directly and predictably associated with the attack.

Impact Score	Examples
 High	Halting of the chain; loss, locking, or unauthorized withdrawal of funds of many users; arbitrary transaction execution; forging of user messages / circumvention of authorization logic
 Medium	Temporary denial of service / substantial unexpected delays in processing user requests (e.g. many hours/days); loss, locking, or unauthorized withdrawal of funds of a single user / few users; failures during transaction execution (e.g. out of gas errors); substantial increase in node computational requirements (e.g. 10x)
 Low	Transient unexpected delays in processing user requests (e.g. minutes/a few hours); Medium increase in node computational requirements (e.g. 2x); any kind of problem that affects end users, but can be repaired by manual intervention (e.g. a special transaction)
 None	Small increase in node computational requirements (e.g. 20%); code inefficiencies; bad code practices; lack/incompleteness of tests; lack/incompleteness of documentation

Exploitability Score

The Exploitability score reflects the ease and technical means by which the vulnerability can be exploited; it represents the characteristics of the vulnerable component. In the below table we list, for each category, examples of actions by actors that are enough to trigger the exploit. In the examples below:

- *Actors* can be any entity that interacts with the system: other blockchains, system users, validators, relayers, but also uncontrollable phenomena (e.g. network delays or partitions).
- *Actions* can be
 - *legitimate*, e.g. submission of a transaction that follows protocol rules by a user; delegation/redelegation/bonding/unbonding; validator downtime; validator voting on a single, but alternative block; delays in relaying certain messages, or speeding up relaying other messages;
 - *illegitimate*, e.g. submission of a specially crafted transaction (not following the protocol, or e.g. with large/incorrect values); voting on two different alternative blocks; alteration of relayed messages.
- We employ also a *qualitative measure* representing the amount of certain class of power (e.g. possessed tokens, validator power, relayed messages): *small* for < 3%; *medium* for 3-10%; *large* for 10-33%, *all* for >33%. We further quantify this qualitative measure as relative to the largest of the system components. (e.g. when two blockchains are interacting, one with a large capitalization, and another with a small

capitalization, we employ *small* wrt. the number of tokens held, if it is small wrt. the large blockchain, even if it is large wrt. the small blockchain)

Exploitability Score	Examples
 High	illegitimate actions taken by a small group of actors; possibly coordinated with legitimate actions taken by a medium group of actors
 Medium	illegitimate actions taken by a medium group of actors; possibly coordinated with legitimate actions taken by a large group of actors
 Low	illegitimate actions taken by a large group of actors; possibly coordinated with legitimate actions taken by all actors
 None	illegitimate actions taken in a coordinated fashion by all actors

Severity Score

The severity score combines the above two sub-scores into a single value, and roughly represents the probability of the system suffering a severe impact with time; thus it also represents the measure of the urgency or order in which vulnerabilities need to be addressed. We assess the severity according to the combination scheme represented graphically below.

As can be seen from the image above, only a combination of high impact with high exploitability results in a Critical severity score; such vulnerabilities need to be addressed ASAP. Accordingly, High severity score receive vulnerabilities with the combination of high impact and medium exploitability, or medium impact, but high exploitability.

Severity Score	Examples
 Critical	Halting of chain via a submission of a specially crafted transaction
 High	Permanent loss of user funds via a combination of submitting a specially crafted transaction with delaying of certain messages by a large portion of relayers
 Medium	Substantial unexpected delays in processing user requests via a combination of delaying of certain messages by a large group of relayers with coordinated withdrawal of funds by a large group of users
 Low	2x increase in node computational requirements via coordinated withdrawal of all user tokens
 Informational	Code inefficiencies; bad code practices; lack/incompleteness of tests; lack/incompleteness of documentation; any exploit for which a coordinated illegitimate action of all actors is necessary

Disclaimer

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability, etc.) set forth in the associated Services Agreement. This report provided in connection with the Services set forth in the Services Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement.

This audit report is provided on an “as is” basis, with no guarantee of the completeness, accuracy, timeliness or of the results obtained by use of the information provided. Informal has relied upon information and data provided by the client, and is not responsible for any errors or omissions in such information and data or results obtained from the use of that information or conclusions in this report. Informal makes no warranty of any kind, express or implied, regarding the accuracy, adequacy, validity, reliability, availability or completeness of this report. This report should not be considered or utilized as a complete assessment of the overall utility, security or bugfree status of the code.

This audit report contains confidential information and is only intended for use by the client. Reuse or republication of the audit report other than as authorized by the client is prohibited.

This report is not, nor should it be considered, an “endorsement”, “approval” or “disapproval” of any particular project or team. This report is not, nor should it be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts with Informal to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor does it provide any indication of the client’s business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should it be leveraged as investment advice of any sort.

Blockchain technology and cryptographic assets in general and by definition present a high level of ongoing risk. Client is responsible for its own due diligence and continuing security in this regard.